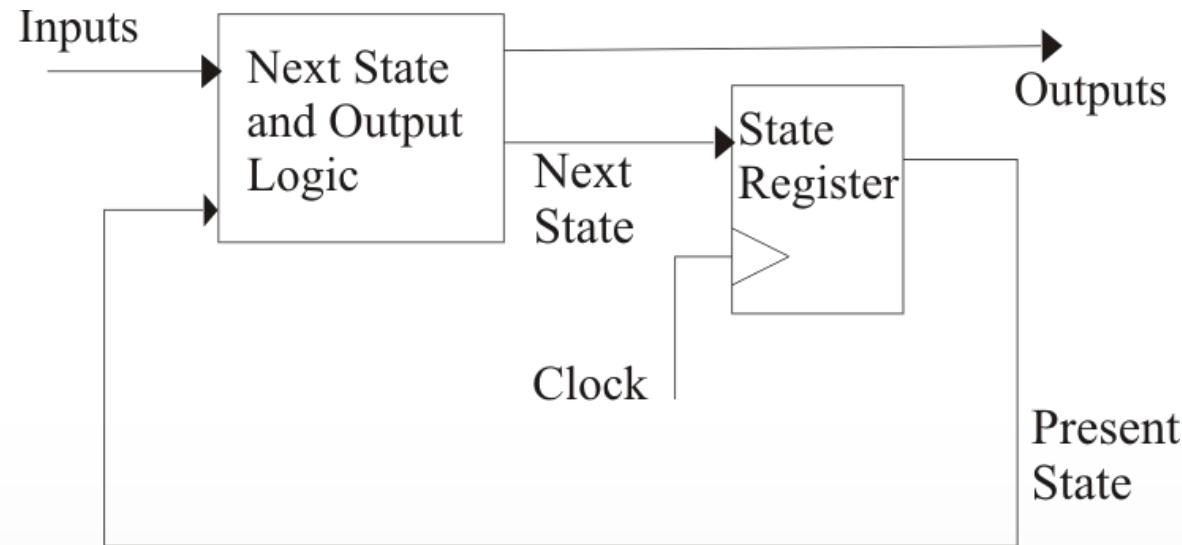


11: State Machines

SystemVerilog Model of State Machine



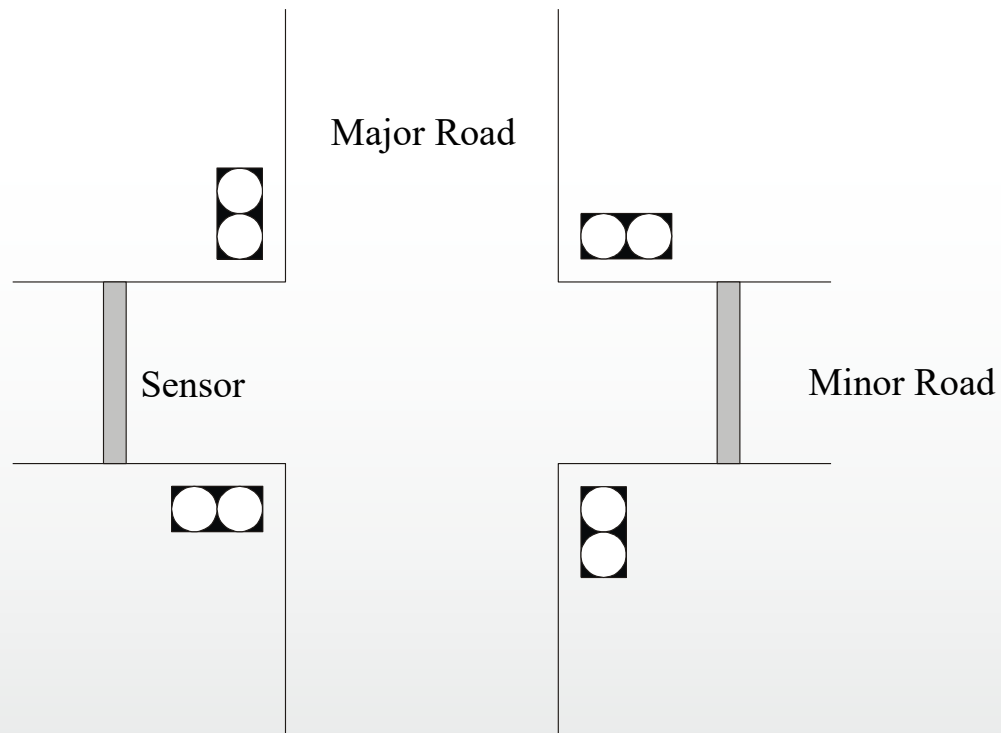
- SystemVerilog describes concurrent hardware
 - Each always block describes a piece of hardware
 - Therefore we can have more than one always block in a module
 - This model has one combinational block (Next State and Output) and one sequential (State Register)

SystemVerilog Model of State Machine

- Use an enumerated type for the states
 - Don't need to do a state assignment
- Combinational part is modelled with `always_comb`
 - Assign to next state and to outputs
 - Case statement – one branch for each state
 - If statements in each branch for Mealy (conditional) outputs and for selecting next state
 - Default assignments to each output and next state at start, to avoid accidental memory
- Sequential part is modelled with `always_ff`
 - Just like a flip-flop

State Machine Example

- Traffic Light Controller



Lights can be:

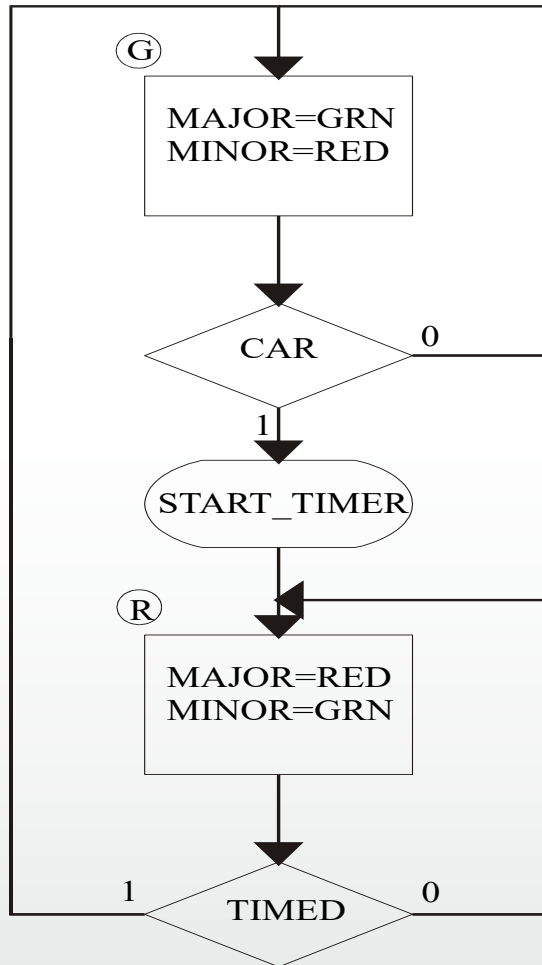
Major Road Red
Minor Road Green

or:

Major Road Green
Minor Road Red

2 states

ASM Chart



Two states (G and R)

Two inputs (CAR and TIMED)

Three outputs

(Minor=Green/Major=Red;
Major=Green/Minor=Red;
Start_timer)

```
module traffic (output logic start_timer,  
               major_green, minor_green,  
               input logic clock, n_reset, timed, car);  
  
enum {G, R} present_state, next_state;  
  
always_ff @(posedge clock, negedge n_reset)  
  begin: SEQ //label  
    if (!n_reset)  
      present_state <= G;  
    else  
      present_state <= next_state;  
  end
```

always_comb

begin: COM

start_timer = '0;

minor_green = '0;

major_green = '0;

next_state = present_state;

unique case (present_state)

G: **begin**

major_green = '1;

if (car)

begin

start_timer = '1;

next_state = R;

end

end

R: **begin**

minor_green = '1;

if (timed)

next_state = G;

end

endcase

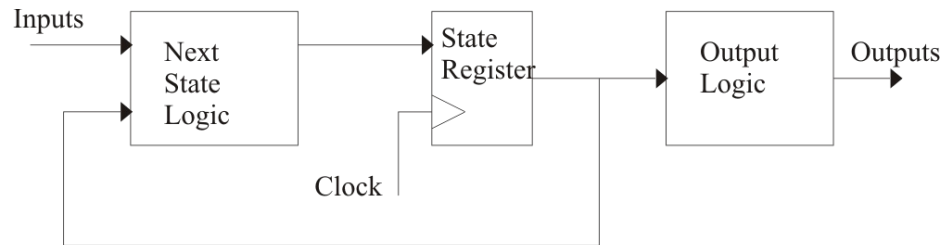
end

endmodule

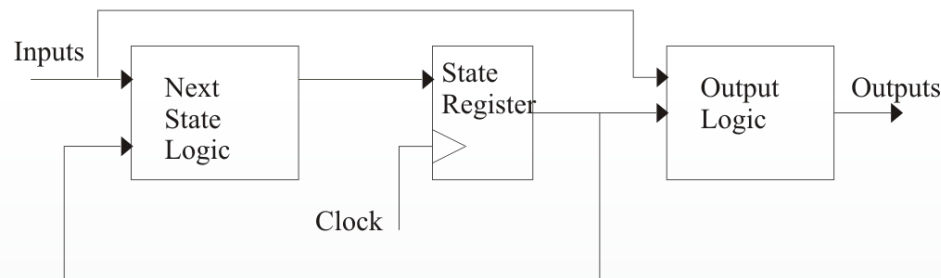
Comments

- present_state and next_state must be variables of enumerated type
- Care is needed with combinational process - can easily create latches - *BAD!*
 - Default values for outputs and next_state
- always blocks are labelled – need begin and end.
- unique case checks that all states are included
- There are other ways to do this, but this is recommended.

Other models



Moore Machine



Mealy Machine

- Separate always_comb for Next State and Output
- One always_ff for everything (but outputs will be flip-flops!)

Design of Vending Machine

- A drink costs 40c. The machine accepts 20c and 10c coins (all other coins are rejected by the mechanics of the system).
- Once 40c have been inserted, the drink is dispensed. If more than 40c are inserted, all coins are returned.
- The machine has two lights: one to show that it is ready for the next transaction, and one to show that further coins need to be inserted.

```

graph TD
    C((C)) --> CoinC[Coin]
    CoinC --> 20cC{20c}
    20cC -- Y --> 10cC{10c}
    20cC -- N --> 10cC
    10cC -- Y --> fC((f))
    10cC -- N --> Merge(( ))
    D((D)) --> CoinD[Coin]
    CoinD --> 20cD{20c}
    20cD -- Y --> Merge
    20cD -- N --> 10cD{10c}
    10cD -- Y --> fD((f))
    10cD -- N --> Merge
    F((F)) --> CoinF[Coin]
    CoinF --> 20cF{20c}
    20cF -- Y --> I((I))
    20cF -- N --> 10cF{10c}
    10cF -- Y --> Merge
    10cF -- N --> Merge
    I --> Return[Return]
    Merge --> Exit(( ))
  
```

SystemVerilog Code

```
module vending(output logic ready, dispense, ret, coin
               input logic clock, n_reset, twenty, ten);

    enum {A, B, C, D, F, I} present_state, next_state;

    always_ff @(posedge clock, negedge n_reset)
    begin: SEQ
        if (~n_reset)
            present_state <= A;
        else
            present_state <= next_state;
    end
```

```
always_comb
begin: COM
ready = '0;
dispense = '0;
ret = '0;
coin = '0;
next_state = present_state;
unique case (present_state)
A : begin
    ready = '1;
    if (twenty)
        next_state = D;
    else if (ten)
        next_state = C;
    end
B : begin
    dispense = '1;
    next_state = A;
    end
C : begin
    coin = '1;
    if (twenty)
        next_state = F;
    else if (ten)
        next_state = D;
    end
```

```
D : begin
    coin = '1;
    if (twenty)
        next_state = B;
    else if (ten)
        next_state = F;
    end
F : begin
    coin = '1;
    if (twenty)
        next_state = I;
    else if (ten)
        next_state = B;
    end
I : begin
    ret = '1;
    next_state = A;
    end
endcase
end
endmodule
```

SystemVerilog – Comments

- present_state and next_state must be variables of enumerated type
- Care is needed with combinational process - can easily create latches
 - Default values for outputs and next_state
- always blocks are labelled – need begin and end.
- Use blocking assignment (=) in always_comb;
use nonblocking assignment (<=) in always_ff.
 - NEVER mix them!
 - This avoids race conditions and indeterminate simulations
- unique case checks that all states are included
- There are other ways to write state machines, but this style is recommended.

State Encoding

State Encoding

- Quartus uses one-hot encoding by default

```
enum {s0, s1, s2} state;
```

will map s0 to 001, s1 to 010 and s2 to 100.

- To force sequential encoding, change to:

```
typedef enum {s0, s1, s2} state_type;  
(* syn_encoding="sequential" *)  
state_type state;
```

State encoding will now be s0=00, s1=01, s2=10.

State Encoding

- Verilog standard defines a different form:

```
(* synthesis, fsm_state="sequential" *)
```

- Might be recognised by synthesis tools

Initial Values

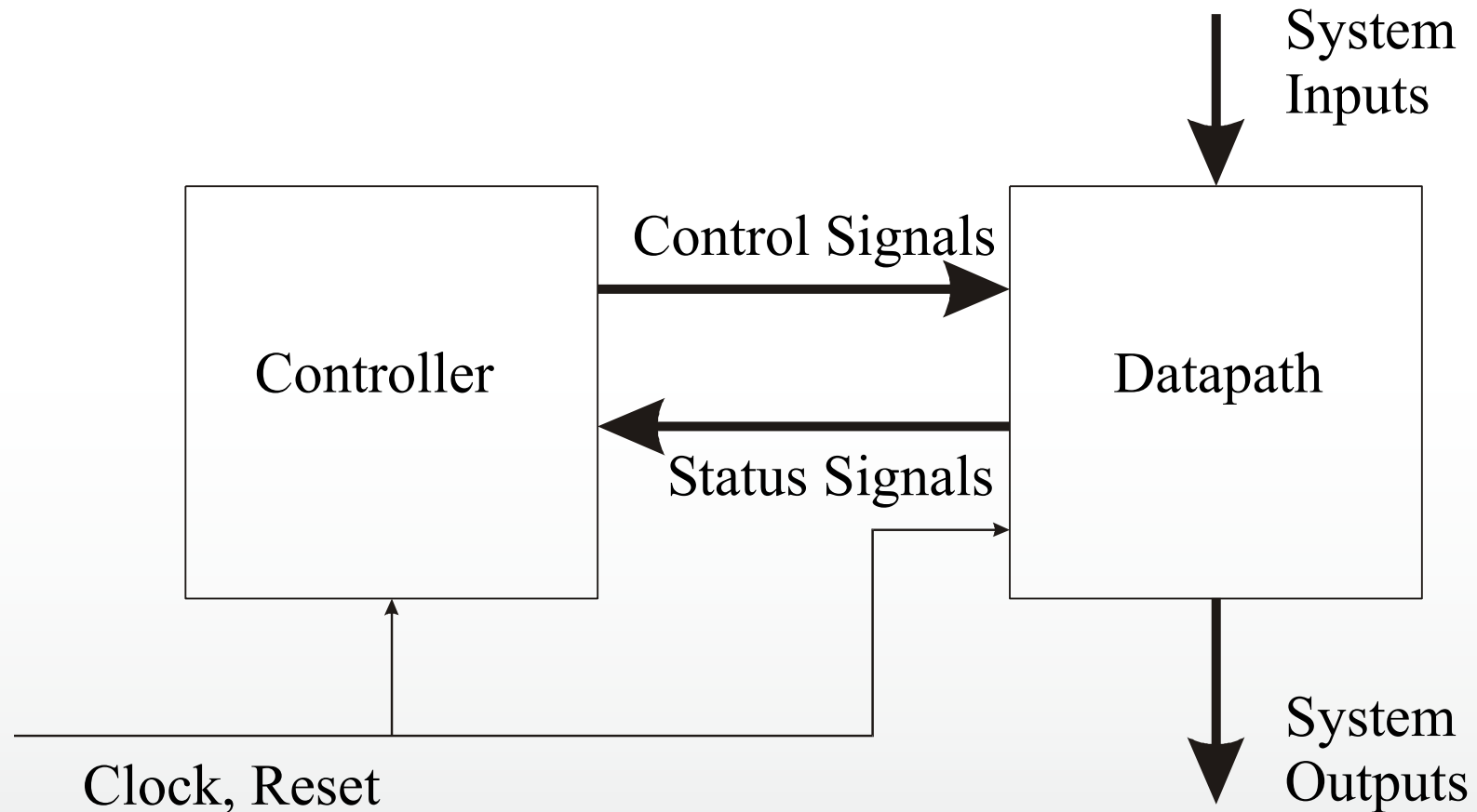
- The initial value of a logic variable is 'X. Shows in simulation as a red line. Useful for debugging.
- (In real life, an 'X or a 'Z will always resolve to '1 or '0.)
- Initial value of an enumerated type is the 'left-hand' value. i.e. A in the example. Less useful for debugging.
- `enum logic [2:0] {A, B, C, D, F, I}`
`present_state, next_state;`
- Initial value (*in simulation*) for present_state and next_state are now 3'bXXX
 - Can easily see if the state machine is not reset.

Datapath/Controller

Sequential Systems

- Any sequential system can be thought of as a state machine
 - ARM Processor? Not a helpful way to think of it.
- Better to think of a state machine controlling other sequential hardware
 - Controller/datapath
 - Division between the 2 is arbitrary (the designer's choice)

Datapath/Controller

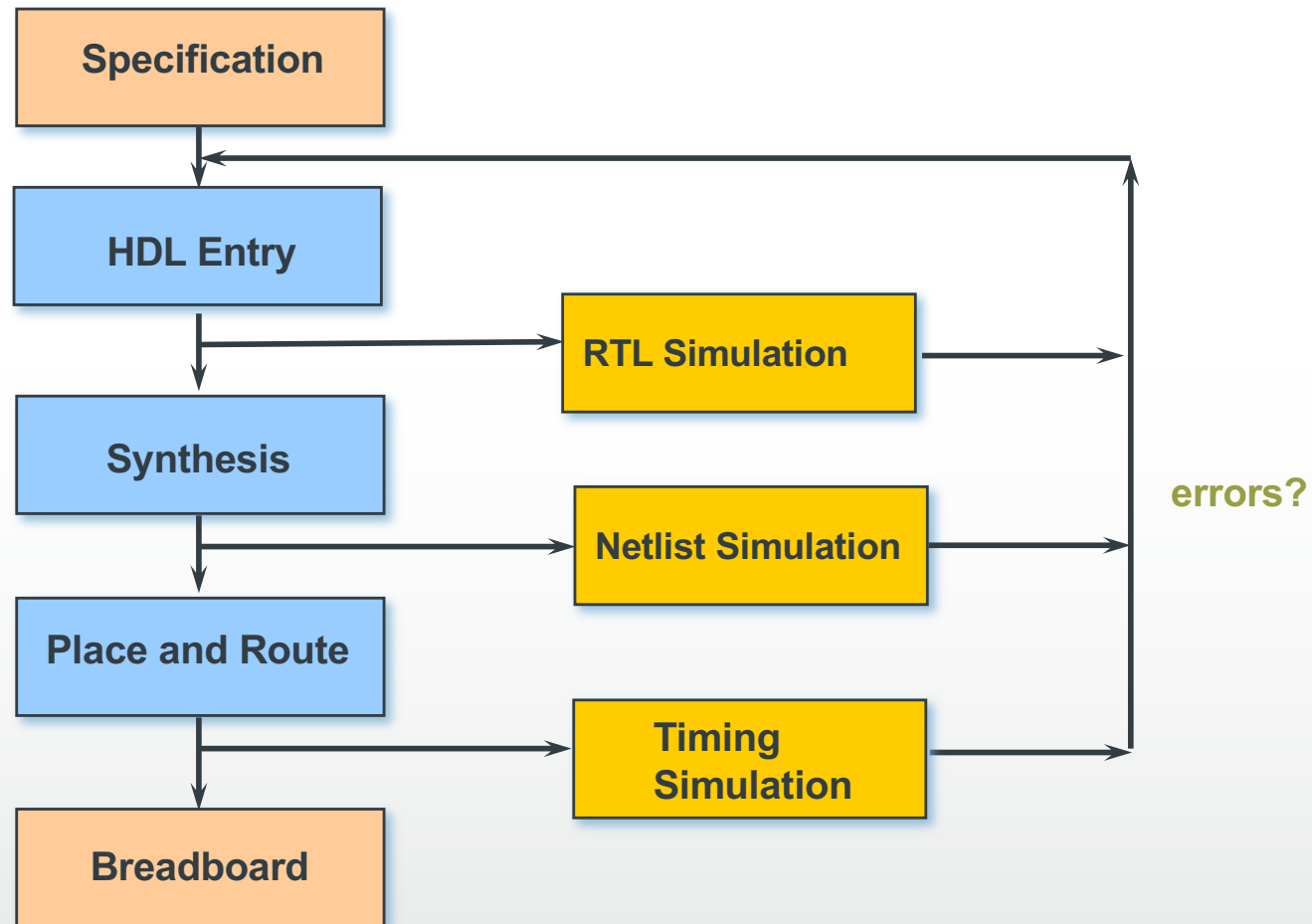


Testbenches for Sequential Systems

Testbenches for Sequential Systems

- Verify synthesisable models by simulation
 - Stimulus generation
 - Response checking
- Verify timing characteristics of simulation model
- Characteristics
 - No ports – entire world to simulation model
 - Instantiate simulation model

HDL Design Flow



Simple Example

```
module test_dff;  
    logic d, q, clk;  
  
    dff d0 (.*); // only works if  
    // internal signals have the same names  
initial  
    begin  
        #10ns d = '0; clk = '0;  
        #10ns clk = '1;  
        #10ns d = '1;  
        #10ns clk = '0;  
    end  
endmodule
```

Clock Generation

```
logic clock;  
always  
  begin  
    #5ns clock = '0;  
    #5ns clock = '1;  
  end
```

- What is initial value of clock?

Clock Generation (2)

```
logic clock;
```

```
initial
```

```
  begin
```

```
    clock = '0;
```

```
    forever #5ns clock = ~clock;
```

```
  end
```

- Mark/space ratio is 1:1

Reset

```
initial  
  begin  
    n_reset = '1;  
    #10ns n_reset = '0;  
    #10ns n_reset = '1;  
  end
```

- Don't start in the reset state
- Can generate more than one signal from a process, but never generate one signal from two processes

Testbench for State Machine

```
module testvending;
```

```
logic ready, dispense, ret,  
       coin, clock, n_reset,  
       twenty, ten;
```

```
vending sm0 (.*);
```

```
initial  
  begin  
    clock = '0;  
    forever  
      #5ns clock = ~clock;  
  end
```

```
initial  
  begin
```

```
    n_reset = '1;  
    #10ns n_reset = '0;  
    #10ns n_reset = '1;
```

```
  end
```

```
initial  
  begin
```

```
    twenty = '0; ten = '0;  
    #30ns twenty = '1;  
    #10ns twenty = '0; ten = '1;
```

```
  end
```

```
endmodule
```